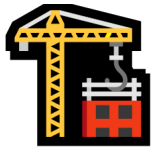
IPT Data Platform Demo

Understanding ELT, dbt, and Metabase

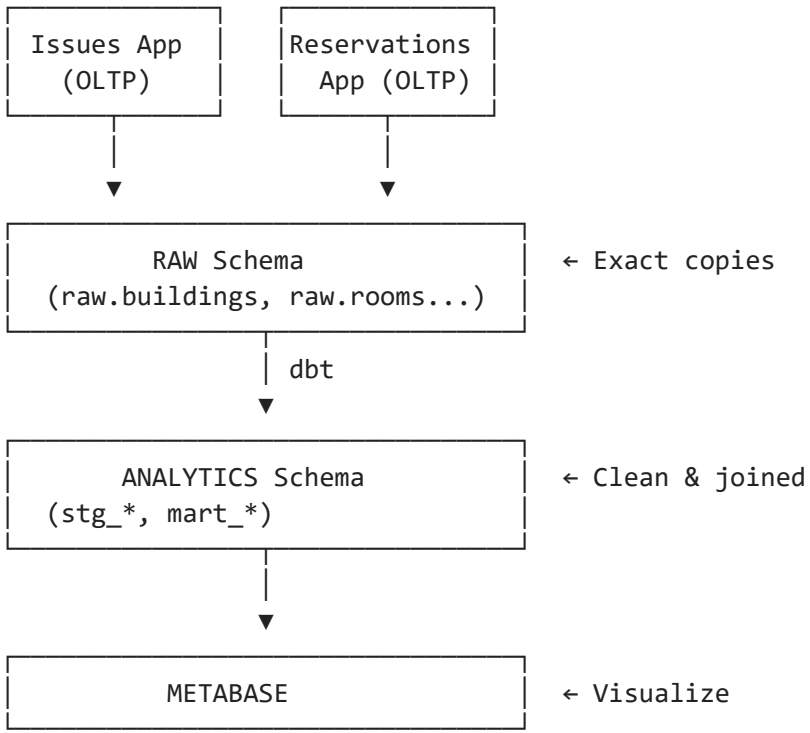
Big Data Course

What We Built 

A **mini data platform** with real tools used in industry:

Layer	Tool	Purpose
Source Systems	PostgreSQL	Simulate operational apps
Ingestion	Bash + psql	Extract & Load data
Transformation	dbt	Clean & model data
Visualization	Metabase	Dashboards & exploration

The Data Flow



Why ELT? (not ETL)

ETL (old way): Transform data *before* loading

ELT (modern way): Load first, transform *inside* the warehouse

Advantages of ELT:

- ✅ Keep raw data for auditing
- ✅ Re-run transforms without re-extracting
- ✅ Use SQL (everyone knows it!)
- ✅ Version control your transformations

The RAW Layer 📦

What: Exact copies of source data + metadata

Where: `raw` schema in warehouse

```
-- Example: raw.issues
SELECT * FROM raw.issues LIMIT 3;

-- Notice the extra columns:
-- _source: 'issues_db' (where it came from)
-- _loaded_at: '2025-12-15 17:00' (when loaded)
```

> **Tip**
>
> **Rule**: Never modify raw data. It's your "source of truth".

Staging Models (stg_*) 🧹

What: Cleaned and standardized data

Purpose:

- Rename columns to consistent style
- Cast data types
- Filter out invalid records
- One stg_ model per source table

```sql
-- stg_issues.sql (simplified)
SELECT
    id AS issue_id,           -- Consistent naming
    room_id,
    UPPER(priority) AS priority,  -- Standardize
    created_at::date AS created_date
FROM {{ source('raw', 'issues') }}
WHERE status IS NOT NULL      -- Filter invalid
```

Mart Models (mart_*) 📊

What: Business-ready analytics tables

Purpose:

- Join multiple sources together
- Calculate metrics and KPIs
- Answer specific business questions
- Ready for dashboards!

```
-- mart_rooms.sql joins:
-- stg_rooms + stg_issues + stg_reservations
-- → Produces: room health status, utilization %, open issues count
```

Our Data Models

Model	Type	Description
`stg_issues`	Staging	Cleaned issues data
`stg_rooms`	Staging	Cleaned rooms data
`stg_reservations`	Staging	Cleaned reservations
`mart_rooms`	Mart	Room health & utilization
`mart_issues`	Mart	Issue details with context
`mart_buildings`	Mart	Building-level summary

> **Note**
>
> **Naming convention**: `stg_` = staging, `mart_` = business mart

Deep Dive: The dbt Pipeline 🔧

Let’s trace data from **source** → **raw** → **staging** → **mart**

We’ll use the **Issues** data as our example.

Step 1: Source Database (init.sql)

This is our **operational app** - where users report issues:

```sql
-- source_databases/issues_db/init.sql
CREATE TABLE issues (
    id SERIAL PRIMARY KEY,
    room_code VARCHAR(10) NOT NULL,
    building_code VARCHAR(5),
    category VARCHAR(50) NOT NULL,
    priority VARCHAR(20) DEFAULT 'medium',
    title VARCHAR(200) NOT NULL,
    status VARCHAR(20) DEFAULT 'open',
    reported_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    resolved_at TIMESTAMP
);
```

> **Important**
>
> This is **OLTP** data - optimized for inserts/updates, not analytics!

Step 2: Ingestion (ingest_docker.sh)

The script copies data **as-is** into the warehouse:

```
# Extract from source
psql "$ISSUES_DB" -c "COPY issues TO STDOUT CSV HEADER"

# Load into warehouse raw schema
psql "$WAREHOUSE" -c "
  COPY raw.issues FROM STDIN CSV HEADER
"
```

Result: `raw.issues` = exact copy + metadata columns

Step 3: Tell dbt Where Data Lives (sources.yml)

dbt needs to know about our raw tables:

```yaml
# dbt/models/sources.yml
version: 2
sources:
  - name: raw
    schema: raw
    tables:
      - name: buildings
      - name: issues        # ← Our raw issues table
      - name: rooms
      - name: reservations
```

Now dbt can reference: `{ source('raw', 'issues') }`

Step 4: Staging Model (stg_issues.sql)

Clean and standardize the raw data:

```sql
-- dbt/models/staging/stg_issues.sql
SELECT
    id as issue_id,              -- Consistent naming
    room_code,
    building_code,
    LOWER(TRIM(category)) as category,  -- Normalize
    LOWER(TRIM(priority)) as priority,
    LOWER(TRIM(status)) as status,
    title,
    reported_at,
    resolved_at,

    -- Add calculated field
    CASE
        WHEN resolved_at IS NOT NULL
        THEN (resolved_at - reported_at) / 3600.0
        ELSE NULL
    END as resolution_hours

FROM {{ source('raw', 'issues') }}  -- Reference raw table
```

Step 5: Mart Model (mart_issues.sql)

Business-ready table with analytics fields:

```sql
-- dbt/models/marts/mart_issues.sql
SELECT
    issue_id,
    room_code,
    building_code,
    category,
    priority,
    status,
    reported_at,
    resolution_hours,

    -- Add time dimensions for filtering
    DATE_TRUNC('month', reported_at) as reported_month,

    -- Calculate age for open issues
    CASE
        WHEN status NOT IN ('resolved', 'closed')
        THEN CURRENT_DATE - reported_at::date
    END as age_days

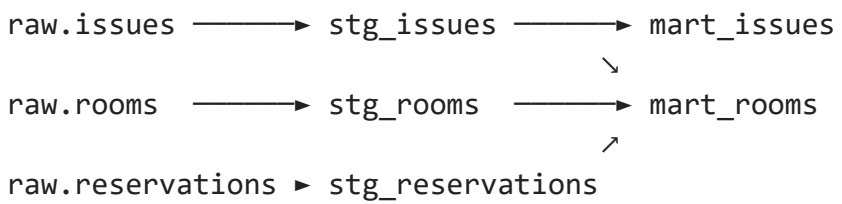
FROM {{ ref('stg_issues') }}  -- Reference staging model
```

The Key dbt Functions

Function	Purpose	Example
`{ source('schema', 'table') }`	Reference raw tables	`{ source('raw', 'issues') }`
`{ ref('model_name') }`	Reference another model	`{ ref('stg_issues') }`

> **Tip**
>
> Using `ref()` creates a **dependency graph** - dbt knows what order to run models!

dbt Builds a DAG (Dependency Graph)



When you run `dbt run`, it:

1. Parses all models
2. Builds dependency graph
3. Runs in correct order (staging first, then marts)

The dbt Project Structure

```
dbt/
├── dbt_project.yml      # Project config
├── profiles.yml         # Database connections
└── models/
    ├── sources.yml      # Define raw tables
    ├── schema.yml       # Documentation & tests
    ├── staging/
    │   ├── stg_issues.sql
    │   ├── stg_rooms.sql
    │   └── stg_reservations.sql
    └── marts/
        ├── mart_issues.sql
        ├── mart_rooms.sql
        └── mart_buildings.sql
```

Running dbt

```
# Run all models
docker compose exec tools dbt run

# Output:
# 1 of 6 START stg_issues..... [RUN]
# 2 of 6 START stg_rooms..... [RUN]
# 3 of 6 START stg_reservations..... [RUN]
# 4 of 6 START mart_issues..... [RUN]
# 5 of 6 START mart_rooms..... [RUN]
# 6 of 6 START mart_buildings..... [RUN]
# Completed: PASS=6 ERROR=0
```

Each model becomes a **table or view** in the `analytics` schema!

Why This Matters 🎯

Without dbt	With dbt
SQL scripts scattered everywhere	Organized folder structure
No dependency management	Automatic execution order
Hard to test	Built-in testing framework
No documentation	Self-documenting models
“It works on my machine”	Version controlled, reproducible

Metabase: What You're Seeing 👀

When you opened Metabase, it **auto-discovered** your tables!

Those "insights" are **X-rays** - automatic explorations of your data.

The Key Sections:

- **Home**: Recent items, auto-generated insights
- **Browse** → **Databases**: See all tables
- **+ New**: Create questions, dashboards
- **Collections**: Organize saved work

Hands-On: Browse Your Data

1. Click **Browse** (left sidebar) → **Databases**
2. Select **IPT Warehouse** → **Analytics**
3. You'll see: `mart_rooms`, `mart_issues`, `mart_buildings`, `stg_*`

Try clicking on `mart_rooms` to see the data!

Hands-On: Create a Simple Chart

Step 1: New Question

Click **+ New** → **Question**

Step 2: Select Data

- Pick **IPT Warehouse** → **Analytics** → **Mart Rooms**

Step 3: Visualize

- Click **Visualize** to see a table
- Click chart icon → **Bar**
- Set X = `room_code`, Y = `open_issues`

Step 4: Save

- Click **Save** → Name it “Issues by Room”

Try These Queries

1. Which rooms have problems?

```
Table: mart_rooms
Filter: health_status = 'critical' OR health_status = 'warning'
```

2. Issues by building

```
Table: mart_buildings
Chart: Bar (building_code vs total_issues)
```

3. Most used rooms

```
Table: mart_rooms
Sort by: total_reservations_90d (descending)
```

Metabase SQL Mode

For more control, use **native SQL**:

1. **+ New → SQL Query**
2. Select **IPT Warehouse**
3. Write your query:

```sql
SELECT
    room_code,
    open_issues,
    utilization_pct_90d
FROM analytics.mart_rooms
WHERE open_issues > 0
ORDER BY open_issues DESC;
```

4. Click **Run** (▶)

Dashboard Example

Combine multiple questions into a **Dashboard**:

1. **+ New** → **Dashboard**
2. Name it “IPT Room Status”
3. Click **+** to add saved questions
4. Arrange and resize cards
5. **Save**

Suggested cards:

- Room health table
- Issues by building (bar chart)
- Total open issues (number card)

Summary: The Layers

Layer	Schema	Models	Purpose
Raw	`raw`	tables	Untouched source data
Staging	`analytics`	`stg_*`	Cleaned, standardized
Marts	`analytics`	`mart_*`	Business-ready metrics
Dashboards	Metabase	-	Visual insights

Learn More

dbt

- dbt Best Practices

Metabase

- Metabase Learn

Quick Reference Card

Useful Commands

```
# Run the full pipeline
docker compose exec tools bash /scripts/run_pipeline.sh

# Just run dbt transforms
docker compose exec tools dbt run

# Test data quality
docker compose exec tools dbt test

# Query warehouse directly
docker compose exec tools psql "$WAREHOUSE" \
  -c "SELECT * FROM analytics.mart_rooms;"
```

URLs

- **Adminer**: http://localhost:8080
- **Metabase**: http://localhost:3000